Témoins numériques et génération causale

1. Deux chantiers effectivement ouverts

15 octobre 2026, édition 2.3. Ce texte accompagne le plan 11 et amende le candidat P1-D 0.3 en 0.4. Il ajoute des choix documentaires pour P1-A et une proposition théorique séparée P1-B0. Aucun réseau, tête prédictive ou tokenizer appris n'a été implémenté ou entraîné. Les vérifications exécutées sont arithmétiques, sur des exemples construits et explicitement non marchands.

P1-A demande si une écriture du passé facilite l'apprentissage d'une même sortie en prix. P1-B0 demande comment produire une distribution cohérente de trajectoires en prédisant des codes successifs. Les deux expériences n'ont ni la même tête ni le même objectif ; leurs scores internes ne seront pas comparés comme s'ils étaient interchangeables.

Témoin numérique non linéaire

Pour v=x32 dans C ou v=m dans Q, conserver la base N(v)=av+b et ajouter `W2*GELU(W1*v+b1)`. W1 et b1 ont chacun r=16 composantes ; W2 est de taille d par r, sans biais supplémentaire. W2 commence à zéro ; W1 est initialisé indépendamment avec une distribution normale de variance 1 et b1 à zéro.

Les entrées commencent donc comme celles de la base affine. L'addition ne bloque pas structurellement le gradient de W2, mais son arrivée dépend aussi du démarrage de la tête commune décrit plus loin. W1 attend que W2 devienne non nul. Ce délai n'est pas une impossibilité d'apprentissage. Le nombre de paramètres de valeur est `2*d+16*(d+2)=18*d+32`.

Témoin numérique périodique

Ajouter à la même base `W*phi(v)`, avec phi contenant sinus et cosinus de `2*pi*v/T` pour les périodes T=0,5 ; 1 ; 2 ; 4 ; 8 ; 16 ; 32 ; 64. W comporte 16d paramètres et commence à zéro ; les fréquences sont fixes. Le total de valeur est 18d. Ce choix conserve la coordonnée v comme entrée numérique, sans prétendre que l'encodeur appris sera toujours injectif.

Les mêmes deux contrôles sont proposés pour C et Q : huit variantes au total avec C, Q, F+ et H+. Les périodes et r sont des choix initiaux, non optimisés sur des prix. Gorishniy et al., NeurIPS 2022 motivent l'examen de traitements numériques plus expressifs ; les formules ci-dessus sont nos contrôles proposés, pas une reproduction de leurs expériences.

2. Paramètres et comparaison équitable

Le cœur de P1-D reste inchangé. Les contrôles numériques n'ajoutent ni nouveau passé, ni actif, ni statistique. Ils changent seulement la fonction de représentation d'un scalaire déjà présent. Leur coût local et leur capacité changent réellement.

Gabarit	C/Q affine	C/Q périodique	C/Q MLP
L=4, d=128	797 717	799 765	799 797
L=6, d=256	4 747 797	4 751 893	4 751 925
L=8, d=512	25 237 525	25 245 717	25 245 749

Les comptes F+/H+ de 0.3 restent inchangés. Les chiffres ont été vérifiés par calcul entier sans instancier un modèle. Le contraste primaire historique Q-H+ reste déclaré ; son interprétation est limitée à Q affine. Les contrôles étendus évaluent si l'avantage éventuel dépasse ce témoin. Choisir après coup le meilleur concurrent et lui attribuer un test statistique non corrigé serait un autre problème.

Proposition d'initialisation commune

Dans une même seed et une même taille, les variantes partagent le tirage initial du cœur, des métadonnées, de la base numérique et de la tête. Les paramètres entraînés restent ensuite séparés. Matrices ordinaires : normale centrée de variance inverse du nombre d'entrées ; biais nuls ; gains LayerNorm égaux à 1 et biais nuls. LayerNorm epsilon proposé : 10^-5. Embeddings d'actif : normale centrée d'écart-type 0,02. Base numérique : a de même écart-type 0,02 et b nul. Corrections symboliques, W2 du témoin MLP et W du témoin périodique : nulles.

Pour éviter un écart initial artificiel entre les quantiles, les poids de la tête sont nuls. Les biais donnent z=-1,645 ; -1,282 ; -0,674 ; 0 ; 0,674 ; 1,282 ; 1,645, pour chaque horizon. Le premier biais est libre ; les six autres sont les inverses softplus des écarts positifs. Ce sont des constantes arrondies de conception, pas des estimations du marché. Au premier pas, la tête peut apprendre ; le gradient vers le cœur devient possible quand ses poids quittent zéro.

Cette initialisation doit faire l'objet de contrôles futurs de gradients et de sorties finies. Elle n'est pas déclarée optimale. Une init différente serait une sensibilité explicite, avec le même droit de réglage pour chaque concurrent.

3. Budget initial proposé et coût des essais

Le candidat 0.4 propose AdamW avec beta1=0,9, beta2=0,95, epsilon=10^-8, décroissance 0,01 et clipping de norme globale des gradients à 1. La décroissance vise les matrices ordinaires, bases et embeddings, sauf biais et gains de normalisation. Une ligne symbolique jamais visitée et initialisée à zéro doit rester nulle. L'article AdamW justifie la distinction avec une pénalité L2 dans Adam ; il ne justifie pas ces valeurs pour nos prix.

Dropout proposé nul pour le premier contraste. Les trois taux maximaux sont 10^-4, 3*10^-4 et 10^-3. Le taux augmente linéairement pendant 5 % des mises à jour, puis suit un cosinus jusqu'à 10 % de son maximum. La référence numérique initiale est float32 pour le réseau, avec normalisation des données en float64 déjà spécifiée ; précision mixte différée à une étude de coût.

Chaque époque présente une permutation des 2 541 dates d'ajustement, avec les six actifs. Un lot contient au plus 32 dates, soit 192 exemples ; le dernier lot de 13 dates est conservé. Cela donne 80 mises à jour par époque. Une campagne de 30 époques représente 2 400 mises à jour et 457 380 exemples actif-origine par exécution. Les fenêtres comportent 256 vecteurs ; leur répétition ne crée pas de données indépendantes.

Pour chacune des huit variantes et chaque taille étudiée : trois taux, deux seeds de réglage, puis cinq nouvelles seeds pour le taux retenu. Les seeds proposées sont 1103/2207 pour régler, et 3109/4211/5323/6427/7537 pour répéter. La sélection utilise la moyenne du score de développement h1 au checkpoint final, avec poids égal aux actifs. Une égalité stricte retient le plus petit taux. Les checkpoints intermédiaires servent aux diagnostics ; ils ne sont pas sélectionnés secrètement.

Cela représente **88 exécutions**, dont 48 de réglage et 40 de répétition, pour la seule petite taille : 211 200 mises à jour, 40 249 440 présentations actif-origine et 10 303 856 640 vecteurs de contexte. Ce budget est un plan, pas un lancement. Temps, mémoire et calcul devront être mesurés avant exécution complète.

Deux autres tailles porteraient le même plan à 264 exécutions. Un second budget de 90 époques, retuné de la même manière pour les trois tailles, porterait le total à 528. Cette enveloppe illustre pourquoi on commence par qualifier le petit dispositif. Égalité d'exposition n'est pas égalité de calcul ni preuve de convergence.

4. Diagnostics avant interprétation

Une sortie non finie, un masque incorrect ou une différence d'entrée non déclarée interrompt l'exécution concernée et est enregistrée. Si un taux échoue pour une seed de réglage, il est classé non admissible pour cette variante ; s'ils échouent tous, aucun gagnant n'est produit. Une panne matérielle peut être relancée avec les mêmes paramètres et état ; un changement scientifique reçoit une nouvelle version.

Les cinq répétitions finales sont conservées, y compris une défaillance numérique. On ne les remplace pas par de meilleures seeds. Les courbes d'objectif d'ajustement, de développement, les normes et les incidents servent à distinguer instabilité, sous-apprentissage et généralisation. Trente époques ne prouvent pas la convergence. La campagne longue et les grandes tailles restent des hypothèses d'étude, pas une promesse d'amélioration.

Les intervalles par blocs et scores de 0.3 restent exploratoires. Aucun p-value confirmatoire n'est annoncé sur le pseudo-test déjà inspecté. Les répétitions d'entraînement quantifient l'optimisation, sans devenir de nouvelles trajectoires économiques. Le gel final doit encore fixer la décision de poursuite et le test indépendant.

Ce qui n'est pas fermé par 0.4

L'adéquation de ce budget, le coût réel, les sensibilités d'initialisation, la tête croisée H2, une fusion hiérarchique non additive et la puissance de l'évaluation restent ouverts. Le candidat fournit une première proposition exécutable ultérieurement, pas un plan universellement optimal. Les listes historiques de points ouverts sont amendées sans effacer leurs versions.

Pourquoi commencer P1-B maintenant

Le porteur veut apprendre la suite des éléments de marché. Attendre la fin de tous les essais P1-A avant d'écrire ce générateur risquerait de déplacer indéfiniment le cœur du projet. Les sections suivantes définissent une première solution causale sur papier, afin de pouvoir contester sa sémantique et ses difficultés dès cette phase.

P1-B0 garde le passé disponible à l'origine i, une ancre P_i et l'échelle s_i. Il produit une suite de vingt codes après cette origine. Il ne demande pas de prédire les anciennes positions du contexte avec une normalisation calculée plus tard. Son caractère GPT vient de la factorisation autorégressive conditionnelle ; sa tête n'est pas obligatoirement un softmax de mots linguistiques.

5. P1-B0 : état causal et prochain code

Poser `u0=log(P_i)` et `delta=s_i/8`, connus à l'origine. Delta est figé pour les vingt pas. L'état reconstruit initial est u_hat0=u0. À l'apprentissage futur, le code cible pour le pas h serait :

```
k_h=floor((log(P_(i+h))-u_hat_(h-1))/delta+1/2)
```

```
u_hat_h=u_hat_(h-1)+delta*k_h
```

La convention arrondit les demi-entiers vers plus l'infini. Le code zéro conserve exactement l'état. C'est une quantification différente de la grille à milieux du passé P1-A ; elle n'est pas introduite silencieusement dans ses contrastes.

Le calcul de k_h utilise le prix cible seulement pour créer son étiquette, et les seuls codes antérieurs pour l'état précédent. En apprentissage teacher forcing, le prédicteur du pas h reçoit les codes vrais jusqu'à h-1, jamais k_h. En génération, il reçoit les codes qu'il a lui-même produits. Recalculer un état futur depuis un vrai prix encore inconnu serait une fuite.

La loi proposée est `p(k_1,...,k_20|H_i)=produit_h p(k_h|H_i,k_<h)`, H_i étant le passé et les métadonnées connus à i. Après génération, `P_hat_h=exp(u_hat_h)`. Les niveaux 1, 5 et 20 proviennent de la même trajectoire. Une nouvelle origine après observation d'un vrai prix peut recalculer son ancre ; cela constitue une nouvelle prévision.

Avec l'encodeur historique défini ci-dessus, l'erreur de reconstruction en log-prix est au plus delta/2 à chaque pas, sans accumulation, puisque le résidu se mesure depuis l'état reconstruit. **Ce n'est pas une borne de prévision.** Le modèle peut produire une suite de codes très différente de celle du futur réel.

Les vérifications de cette session utilisent des logarithmes rationnels construits pour contrôler la récurrence, notamment les demi-entiers et les signes. Aucun exemple n'est présenté comme une cotation. Le codec théorique repose sur une arithmétique exacte ; sa stabilité float et la gestion des exponentielles devront être étudiées avant développement.

6. Une loi sur tous les entiers, sans tête abandonnée

Une catégorie ESC seule ne définit pas la valeur du futur extrême. Une tête de queue séparée peut aussi ne recevoir aucun exemple. Nous proposons d'étudier une base numérique sur tout entier, utilisée dans la probabilité de chaque cible, puis des corrections dans le dictionnaire.

Pour un état conditionnel donné, soient mu réel et sigma strictement positif. Définir une normale arrondie par :

`q(k)=Phi((k+1/2-mu)/sigma)-Phi((k-1/2-mu)/sigma)`

Les cellules partitionnent la droite : q(k)>0 et la somme sur tous les entiers vaut 1 en arithmétique exacte. Pour I={-2048,...,2047}, poser `w(k)=exp(a_k)` dans I et w(k)=1 ailleurs. La loi corrigée est `p(k)=q(k)*w(k)/Z`.

Le normalisateur ne nécessite pas une somme infinie : `Z=q(hors I)+somme_(k dans I) q(k)*exp(a_k)`. Les deux queues de q se calculent avec la fonction de répartition aux frontières -2048,5 et 2047,5. Cette écriture positive est préférable conceptuellement à une soustraction de nombres proches. L'évaluation numérique stable de log q, des queues et de log Z reste à spécifier.

À a_k=0, p=q. Les paramètres mu et sigma interviennent aussi dans les observations ordinaires : il n'y a plus une tête réservée uniquement à des exemples ESC absents. Cela ne garantit pas que leur apprentissage soit bon ; les corrections peuvent absorber une partie de la structure. L'identifiabilité et les interactions devront être examinées.

Un échantillonnage exact sur papier choisit parmi les cellules centrales et les deux queues agrégées, puis utilise une normale tronquée et l'arrondi dans la queue sélectionnée. Le support demeure non borné : on n'écrête pas les prix extrêmes au bord d'un dictionnaire. Les paramètres des corrections pourront être plats ou hiérarchiques, mais la tête complète n'est pas encore arrêtée.

Le principe de masses obtenues par différence de CDF a notamment une antériorité dans PixelCNN++, section 2.1. Cet article traite des pixels bornés avec des mélanges logistiques ; il ne valide pas notre loi non bornée, le transfert au prix ou les corrections proposées. Aucune nouveauté absolue n'est revendiquée pour cette famille de constructions.

7. Les queues du logarithme ne suffisent pas

Une loi discrète dont la masse de queue positive est asymptotiquement proportionnelle à exp(-k/b) peut sembler régulière. Mais le prix contient `exp(delta*k)`. Son espérance diverge lorsque `delta*b` est supérieur ou égal à 1. Une loi valide sur les codes peut donc produire une espérance de prix infinie. Le passage au prix brut est une partie de la conception probabiliste, pas une simple opération d'affichage.

La normale arrondie proposée évite cette difficulté pour un état conditionnel fixé : ses moments exponentiels sont finis. Cela ne suffit pas automatiquement pour une trajectoire autorégressive, car ses paramètres futurs dépendent des codes générés. Nous donnons une condition suffisante, encore à paramétrer expérimentalement.

Supposons pour tous les états autorisés `|mu|<=M`, `0<sigma<=S` et `|a_k|<=A`, avec M,S,A finis, et delta figé. Une normale Y vérifie K=arrondi(Y)<=Y+1/2. Donc :

```
E_q[exp(delta*K)] <= exp(delta/2+delta*M+delta^2*S^2/2)
```

Comme exp(-A)<=Z<=exp(A), le rapport p/q est au plus exp(2A). Il existe ainsi une borne uniforme C sur `E_p[exp(delta*K)|passé]`, égale au produit des deux bornes. Par espérance conditionnelle itérée : `E[P_hat_h|H_i] <= P_i*C^h`, pour chaque h fini.

Cette démonstration borne les paramètres des distributions, **pas les codes ni les prix générés**. Des transformations tanh et sigmoïdes pourraient imposer ces contraintes ; M,S,A, le plancher de sigma et leur adéquation restent ouverts. La borne peut être très large : existence d'un moment ne signifie ni calibration, ni faible erreur.

Le critère d'apprentissage P1-B0 porterait sur la vraisemblance des codes. On ne l'appellera pas densité des prix continus sans définir la mesure et le traitement à l'intérieur des cellules. Les quantiles, couvertures et scores de trajectoire devront être rapportés en prix, avec l'erreur de quantification et l'incertitude Monte-Carlo.

Décisions issues du panel

Retenir cette loi comme candidat théorique, pas comme architecture gagnante. Conserver les pistes de génération grossier/fin et d'autres familles de distributions. Étudier ensuite les bornes, les gradients, le calcul stable et la comparaison avec un témoin sans corrections. Les résultats de marché restent entièrement à produire.